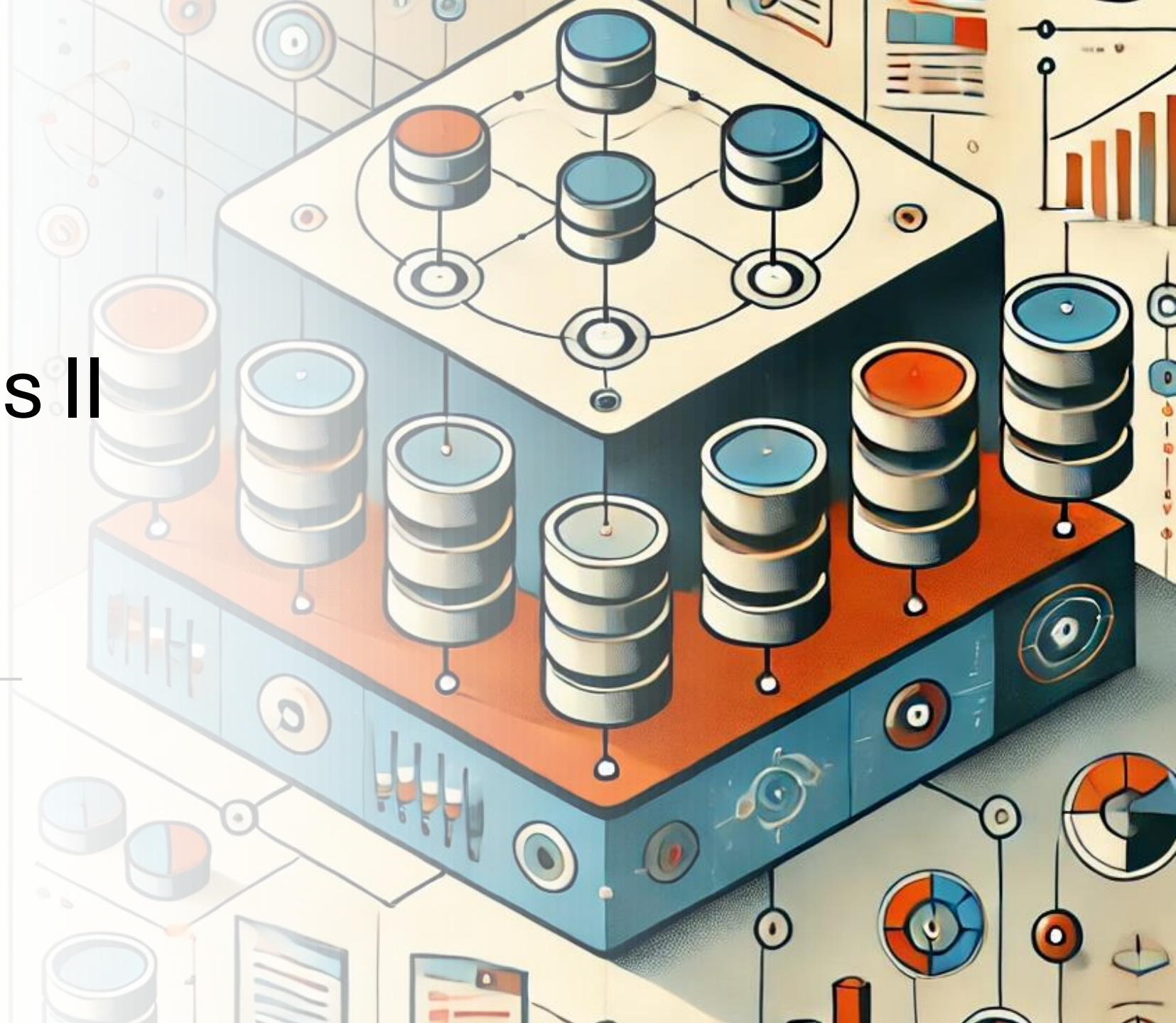


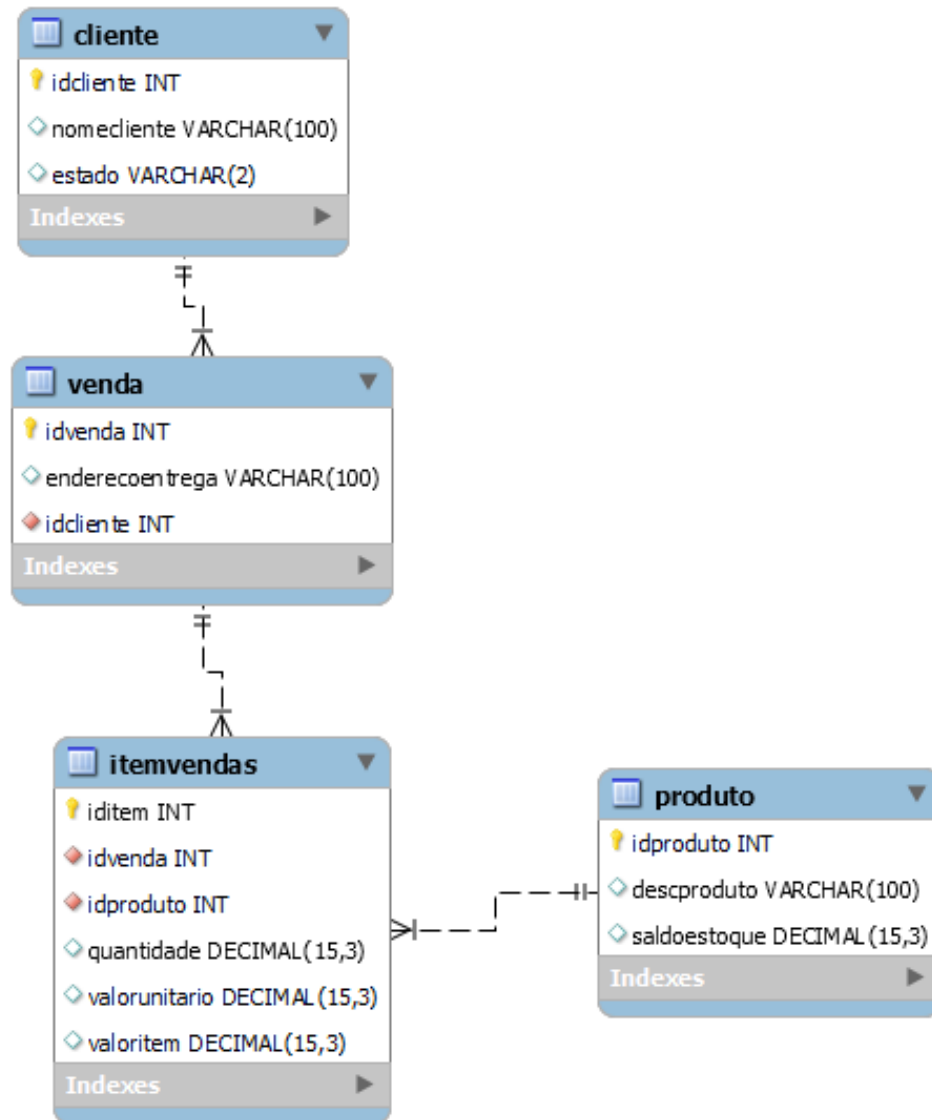
Banco de Dados II

Operadores e Funções

Prof. Me. Jefferson Passerini



Modelo de dados



Consulta com operadores

Operadores aritméticos

Simulação de acréscimo no saldo do estoque (soma)

```
1 -- Active: 1708781131871@@127.0.0.1@5432@bdaula@public PostgreSQL
  ▶ Execute | JSON
2 select idproduto, descproduto, saldoestoque,
3        saldoestoque+100 as "saldo_estoque_acrescimo"
4 from produto; 1ms
```

	idproduto integer	descproduto varchar(100)	saldoestoque numeric	saldo_estoque_acrescimc
1	1	papel higienico	200.000	300.000
2	2	detergente	100.000	200.000
3	3	desinfetante	250.000	350.000
4	4	arroz	300.000	400.000
5	5	feijao	150.000	250.000
6	6	creme de leite	50.000	150.000
7	7	leite longa vida	75.000	175.000

Não altera o saldo real
do produto

Consulta com operadores

Operadores aritméticos

Simulação de acréscimo de valor (%)

```
1 -- Active: 1708781131871@@127.0.0.1@5432@bdaula@public PostgreSQL
  ▶ Execute | JSON
2 select quantidade, valorunitario, valoritem,
3      valorunitario*1.20 as "valorunitario_com_aumento",
4      valoritem*1.20 as "valoritem_com_aumento" from itemvendas;
```

Q	quantidade numeric	valorunitario numeric	valoritem numeric	valorunitario_com_aumei	valoritem_com_aumento
1	10.000	12.000	120.000	14.40000	144.00000
2	6.000	1.500	9.000	1.80000	10.80000
3	3.000	8.000	24.000	9.60000	28.80000
4	2.000	14.000	28.000	16.80000	33.60000
5	3.000	5.000	15.000	6.00000	18.00000
6	12.000	4.000	48.000	4.80000	57.60000
7	2.000	8.000	16.000	9.60000	19.20000

Consulta com operadores

Operadores auxiliares

- **IS NULL** → verifica se um campo é vazio
- **BETWEEN** → verifica um valor numa faixa de valores
- **IN** → verifica se um valor existe na tabela
- **LIKE** → verifica um valor buscando seus semelhantes

Consulta com operadores

Operadores auxiliares

- Inclua novos atributos na tabela produto

▷ Execute

```
alter table produto add column perecivel varchar(1);
```

▷ Execute

```
alter table produto add column valorunitario numeric(15,3);
```

Consulta com operadores

Operadores auxiliares

- **Atualize os dados para o atributo valorunitario**

```
#inclua valores nos produtos
```

```
▷ Execute
```

```
update produto set valorunitario = 10 where idproduto = 1;
```

```
▷ Execute
```

```
update produto set valorunitario = 20 where idproduto = 2;
```

```
▷ Execute
```

```
update produto set valorunitario = 30 where idproduto = 3;
```

```
▷ Execute
```

```
update produto set valorunitario = 40 where idproduto = 4;
```

```
▷ Execute
```

```
update produto set valorunitario = 50 where idproduto = 5;
```

Consulta com operadores

Operadores auxiliares

- Atualize os dados para o atributo perecível

```
#atualize os valores de pericivel nos produtos
```

```
▷ Execute
```

```
update produto set perecivel = 'S' where idproduto = 1;
```

```
▷ Execute
```

```
update produto set perecivel = 'S' where idproduto = 2;
```

```
▷ Execute
```

```
update produto set perecivel = 'S' where idproduto = 4;
```

Consulta com operadores

Operadores auxiliares

- Verifique se existem produtos sem valores unitários

▶ Execute | JSON

```
select * from produto where valorunitario is null;
```

idprodu integer	descproduto varchar(100)	saldoestoque numeric	perecivel varchar(1)	valorunitario numeric
6	creme de leite	50.000	(NULL)	(NULL)
7	leite longa vida	75.000	(NULL)	(NULL)

IS NULL

Consulta com operadores

- Verifique se existem produtos com valores unitários entre 20 e 40

Operadores auxiliares

Execute | JSON

```
select * from produto where valorunitario BETWEEN 20 and 40;
```

idprodu integer	descproduto varchar(100)	saldoestoque numeric	perecivel varchar(1)	valorunitario numeric
3	desinfetante	250.000	(NULL)	30.000
2	detergente	100.000	S	20.000
4	arroz	300.000	S	40.000

BETWEEN

Consulta com operadores

- Selecione os clientes dos estados de SP e MG

Operadores auxiliares

▶ Execute | JSON

```
select * from cliente where estado in ('SP','MG');
```

IN

 idcliente integer	nomecliente varchar(100)	estado varchar(2)
1	Antonio	SP
2	Joao	SP
3	Jose	MG
5	Joaquina	MG

Consulta com operadores

- Selecione os clientes que comecem com a letra A

Operadores auxiliares

▶ Execute | JSON

```
select * from cliente where nomecliente like 'A%'
```

LIKE

 idcliente integer	nomecliente varchar(100)	estado varchar(2)
1	Antonio	SP

Consulta com operadores

- Selecione os clientes que possuem a letra “t” no nome como terceiro caractere

Operadores auxiliares

▶ Execute | JSON

```
select * from cliente where nomecliente like '__t%'
```

LIKE

 idcliente integer	nomecliente varchar(100)	estado varchar(2)
1	Antonio	SP

Consulta com operadores

- Selecione os clientes que possuem “aq” no meio do nome

Operadores auxiliares

▶ Execute | JSON

```
select * from cliente where nomecliente like '%aq%'
```

LIKE

 idcliente integer	nomecliente varchar(100)	estado varchar(2)
5	Joaquina	MG

Consulta com operadores

- Selecione os clientes que terminem em “io”

Operadores auxiliares

```
▶ Execute | JSON  
select * from cliente where nomecliente like '%io'
```

LIKE

 idcliente integer	nomecliente varchar(100)	estado varchar(2)
1	Antonio	SP

Funções

- Definição
- Função é um recurso que facilita muitas tarefas corriqueiras do PostgreSQL, disponibilizando diversas funções divididas em grupos de características operacionais.
- Encontram-se funções de agregação, funções de data e hora, funções de string, entre outras.

Funções

- Sintaxe
- `funcao <expressao>`
- Em que `funcao` é uma das funções internas do programa e `expressao` corresponde o valor avaliado ou atributo da tabela a ser informado.

Funções de Agregação

- As funções de agregação são as ferramentas para obter resultados estatísticos do banco de dados em uso. Estão disponibilizadas as funções:
- **AVG()** retorna a média aritmética
- **COUNT()** retorna a quantidade de registros
- **MAX()** retorna o maior valor
- **MIN()** retorna o menor valor
- **SUM()** retorna a somatória

Funções de Agregação

Operadores auxiliares

- Consulte a média aritmética do valor final dos itens de vendas

▶ Execute | JSON

```
select avg(valoritem) from itemvendas;
```

AVG

avg



37.1428571428571429

Funções de Agregação

Operadores auxiliares

- Verifique quantos item foram vendidos na venda com id=1

▶ Execute | JSON

```
select count(*) from itemvendas where idvenda=1;
```

Count

count

4

Funções de Agregação

Operadores auxiliares

- Verifique qual o item vendido com o maior valor final.

```
▶ Execute | JSON  
select max(valoritem) from itemvendas;
```

Max

Q	max	↕
1	120.000	

Funções de Agregação

Operadores auxiliares

- Verifique qual o item vendido com o menor valor final.

▶ Execute | JSON

```
select min(valoritem) from itemvendas;
```

Q	min	
1	9.000	

Min

Funções de Agregação

Operadores auxiliares

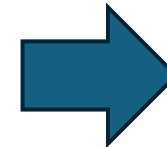
- Some os valores dos itens vendidos da venda id=1

Execute | JSON

```
select sum(valoritem) from itemvendas where idvenda = 1;
```

Sum

iditem integer	idvenda integer	idproduto integer	quantidade numeric	valorunitario numeric	valoritem numeric
1	1	1	10.000	12.000	120.000
2	1	2	6.000	1.500	9.000
3	1	3	3.000	8.000	24.000
4	1	4	2.000	14.000	28.000



Q	sum
1	181.000

Funções de Data e Hora

- As funções de Data e Hora são as ferramentas usadas para se obter acesso a várias características de tempo disponibilizadas no banco de dados. Serão apresentadas apenas as mais comuns:
- **DATE:** função sem parênteses que formata a sequência de caracteres no formato AAAA-MM-DD.
- **AGE():** retorna a diferença de tempo entre duas datas.
- **EXTRACT():** retorna a extração por exemplo do dia, do mês, do ano, da hora, do minuto e do segundo de uma determinada data.
- **NOW():** retorna a data e a hora atual do sistema.

Atualize os dados para datavalidade conforme abaixo.

Funções de Data e Hora

- Altere a tabela produto e insira o campo datavalidade

```
Execute  
alter table produto add column datavalidade DATE;
```

idp integers	descproduto varchar(100)	saldoestoc numeric	perer varchar	valorunita numeric	datavalidade date
1	papel higienico	200.000	S	10.000 numeric	2024-12-31
2	detergente	100.000	S	20.000	2025-07-31
3	desinfetante	250.000	(NULL)	30.000	2025-12-31
4	arroz	300.000	S	40.000	2024-04-30
5	feijao	150.000	(NULL)	50.000	2024-05-31
6	creme de leite	50.000	(NULL)	(NULL)	2024-06-15
7	leite longa vida	75.000	(NULL)	(NULL)	2024-07-10

Funções de Data e Hora

- Verifique quantos meses faltam para o vencimento dos produtos

Execute | JSON

```
select descproduto, datavalidade,  
       (datavalidade - DATE '2024-03-13')/30 as dias_para_vencimento from produto;
```

descproduto varchar(100)	datavalidade date	dias_para_vencimento
papel higienico	2024-12-31	9
detergente	2025-07-31	16
desinfetante	2025-12-31	21
arroz	2024-04-30	1
feijao	2024-05-31	2
creme de leite	2024-06-15	3
leite longa vida	2024-07-10	3

Funções de Data e Hora

- Verifique detalhadamente o tempo para o vencimento dos produtos

▶ Execute | JSON

```
select descproduto, datavalidade,  
       age(datavalidade, '2024-03-13') as anos_meses_dias from produto;
```

descproduto varchar(100)	datavalidad date	anos_meses_dias
papel higienico	2024-12-31	{ "months": 9, "days": 18 }
detergente	2025-07-31	{ "years": 1, "months": 4, "days": 18 }
desinfetante	2025-12-31	{ "years": 1, "months": 9, "days": 18 }
arroz	2024-04-30	{ "months": 1, "days": 17 }
feijao	2024-05-31	{ "months": 2, "days": 18 }
creme de leite	2024-06-15	{ "months": 3, "days": 2 }
leite longa vida	2024-07-10	{ "months": 3, "days": 28 }

Funções de Data e Hora

- Exemplos de EXTRACT() e NOW()

▶ Execute | JSON

```
select extract(HOUR from now()) as hora,  
       extract(MINUTE from now()) as minuto,  
       extract(SECOND from now()) as segundo
```

hora	minuto	segundo
------	--------	---------

15	58	2.048943
----	----	----------

Funções de Data e Hora

- Consulte produtos com vencimento em 2024 entre os meses 4 e 7

▶ Execute | JSON

```
select * from produto  
where extract(year from datavalidade) = 2024  
and extract(month from datavalidade) BETWEEN 4 and 7;
```

idprc	descprc	saldoe	pere	valorunitar	datavalidade
4	arroz	300.000	S	40.000	2024-04-30
5	feijao	150.000	(NULL)	50.000	2024-05-31
6	creme de	50.000	(NULL)	(NULL)	2024-06-15
7	leite longa	75.000	(NULL)	(NULL)	2024-07-10

Funções de String

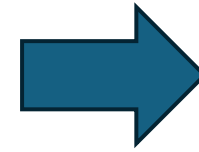
- As funções de string são as ferramentas que possibilitam manipular a estrutura de caracteres definidos no banco de dados.
- **ASCII()** → retorna o código ASCII do string informado
- **||** → retorna a concatenação de trechos de string informados
- **LENGTH()** → retorna o tamanho de uma string informado
- **LOWER()** → retorna em caracteres minúsculos o string informado
- **UPPER()** → retorna em caracteres maiúsculos o string informado

Funções de String

- Acessando a tabela ASCII

▷ Execute | JSON

```
select ascii('a');
```

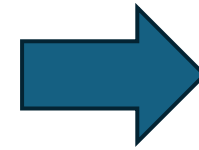


ascii ↕

97

▷ Execute | JSON

```
select ascii('A');
```



ascii ↕

65

Funções de String

- Concatenando Strings

▶ Execute | JSON

```
select 'Joao' || 'Maria' || 'Silva' as nomecompleto
```



nomecompleto

JoaoMariaSilva

Funções de String

- Concatenando Strings

▶ Execute | JSON

```
select descproduto || ' vence em ' || datavalidade as "vencimento" from produto;
```

vencimento
papel higienico vence em 2024-12-31
detergente vence em 2025-07-31
desinfetante vence em 2025-12-31
arroz vence em 2024-04-30
feijao vence em 2024-05-31
creme de leite vence em 2024-06-15
leite longa vida vence em 2024-07-10

Funções de String

- Tamanho de Strings

Execute | JSON

```
select descproduto,  
       length(descproduto) as numero_caracteres  
from produto
```

1ms

descproduto varchar(100)	numero_caracteres
papel higienico varchar(100)	15
detergente	10
desinfetante	12
arroz	5
feijao	6
creme de leite	14
leite longa vida	16

Funções de String

- Maiúsculas e Minúsculas

Execute | JSON

```
select upper(descproduto) as maiusculas,  
       lower(descproduto) as minusculas  
from produto
```

2ms

maiusculas	↕	minusculas
PAPEL HIGIENICO		papel higienico
DETERGENTE		detergente
DESINFETANTE		desinfetante
ARROZ		arroz
FEIJAO		feijao
CREME DE LEITE		creme de leite
LEITE LONGA VIDA		leite longa vida